Automatización del Cambio de Escala en Luxometría.

Departamento de Física, Facultad de Ciencias, Universidad Nacional de Colombia, Bogotá D.C, Colombia

Diaz Chiquillo, Braither. Ortega Perenguez, Edison.
brdiazc@unal.edu.co, eortegap@unal.edu.co

Resumen: El proyecto se enfoca en automatizar el cambio de escala en un luxómetro, mejorando su eficiencia y precisión. A través de un circuito con Arduino, protoboard y motor, se logra la optimización de la medición de iluminancia, implementando piezas impresas en 3D para ensamblar el motor con la perilla del luxómetro. El programa Arduino, respaldado por las bibliotecas EEPROM y Stepper, gestiona modos de operación manual o automático, limitando el movimiento del motor para prevenir daños. A pesar de desafíos en la gestión de errores y la precisión del algoritmo, el proyecto logra la automatización del cambio de escala. Se identifican áreas de mejora en el diseño de piezas e implementación de algoritmos para futuros proyectos en medición de intensidad lumínica.

1. Descripción del Circuito

1.1. Materiales y equipo

Los materiales utilizados para la construcción del dispositivo se muestran a continuación.

- Protoboard
- Arduino UNO
- 1 integrado 74HC14
- Motor de paso 28BYJ-48
- Driver ULN2003
- 2 pulsadores de dos patas
- 1 DIP Switch

El driver ULN2003 es un complemento fundamental para controlar el motor 28BYJ-48. El integrado 74HC14 o Schmitt Trigger será utilizado para prevenir el posible ruido generado por las pulsaciones de los botones.

1.2. Diseño y descripción del circuito

El diagrama del circuito se puede observar en la figura 1 y en la figura 2. El circuito cuenta con dos partes principales. La primera de ellas incluye al motor 28BYJ-48, el driver ULN2003 y el Arduino UNO. La segunda parte incluye los pulsadores, el integrado 74HC14 y el DIP switch.

La función principal de la primera parte del circuito es habilitar el control del motor, mientras que la función principal de la segunda es establecer las diversas funciones para controlar el motor. El dispositivo está construido para tener un switch de encendido y apagado y un switch de modo de operación. El switch de modo de operación tiene dos posibles estados, cuando está desactivado o en '0' se encuentra en el modo manual, mientras que cuando está activado se encuentra en el modo automático. En el modo manual, hay dos pulsadores que permiten el giro en sentido horario o en sentido antihorario. Estos pulsadores están conectados al integrado 74HC14, con la finalidad de prevenir el ruido que puede generarse al realizar la pulsación. De esta forma, un pulso a través del pulsador izquierdo pasa por el integrado y llega al Arduino, allí, este es interpretado como una rotación antihoraria. Ocurre algo similar con el pulsador derecho. En el modo automático, el Arduino recibe una señal a través del programa que genera la respectiva rotación en sentido horario o antihorario según sea necesario.

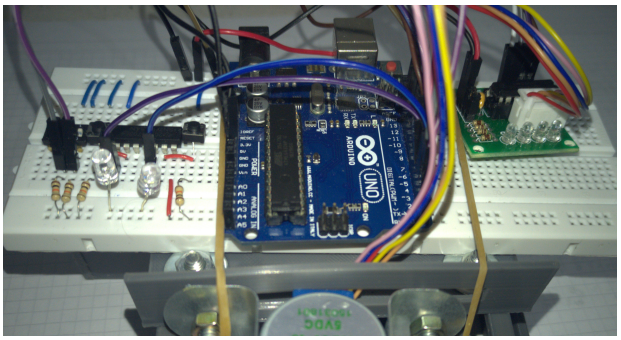


Figura 1. Vista frontal del montaje del circuito del dispositivo.

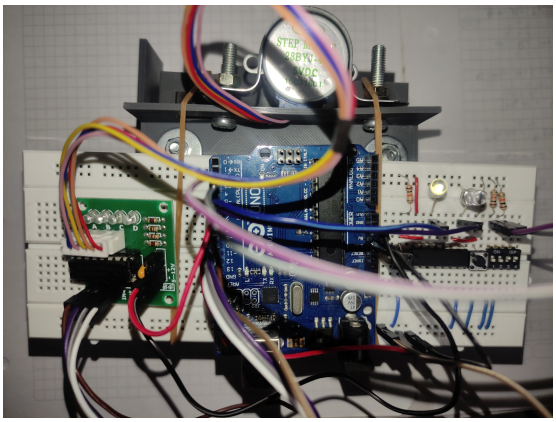


Figura 2. Vista superior del montaje del circuito del dispositivo.

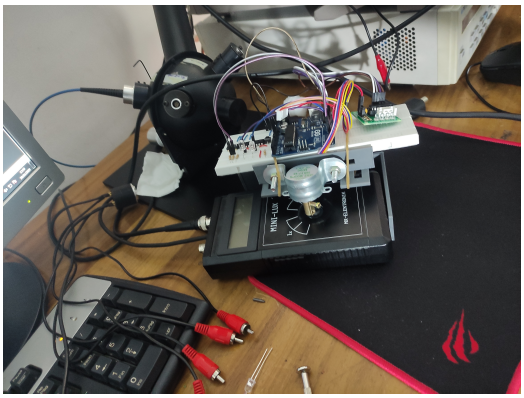


FIGURA 3. Imagen que muestra el ensamble completo de la pieza fabricada, el circuito y el luxómetro

La fabricación de las piezas esenciales para el ensamblaje del circuito demandó el uso de Autocad para su diseño. Este diseño consiste de dos piezas pensadas de manera ajustable, permitiendo lograr una alineación óptima entre el motor y la perilla del luxómetro, como se ilustra en la figura 4: una vez teniendo el diseño de las piezas, se encargó su respectiva impresión en 3D. Para acoplar el motor con la perilla del luxómetro, se empleó una regleta de conexión de 6 mm de diámetro.

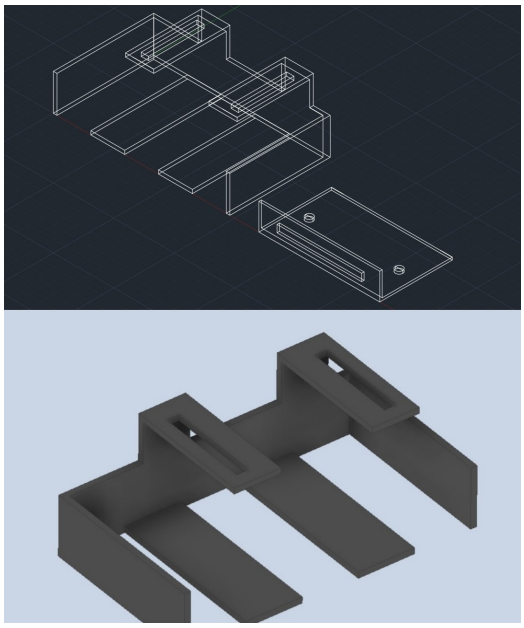


FIGURA 4. Imagen que ilustra el diseño planeado para ubicar el circuito y luxómetro.

2. Código Fuente

```
1
2  #include <EEPROM.h>
3  #include <Stepper.h>
4
5  int StepsPerLoop = 2048;
6  const int MotorSpeed = 6;
7  int MaxGrades = 360;
8  int LuxometerStepGrades = 30;
9  int GeneralPosition;
10
11 Stepper Motor(StepsPerLoop, 8, 10, 9, 11);
12
13 void setup() {
14   pinMode(2, INPUT);
15   pinMode(3, INPUT);
16   pinMode(4, INPUT);
17   pinMode(5, INPUT);
18
19   Motor.setSpeed(MotorSpeed);
20   //EEPROM.write(0, 0); //Activar solo
21   //la primera vez para configurar el valor
22   //inicial del motor.
23   Serial.begin(9600);
24 }
25
26 void loop() {
27   int Power = digitalRead(2);
28   int Mode = digitalRead(3);
29   int L_Rot = digitalRead(4);
30   int R_Rot = digitalRead(5);
31
32
33   if (Power == HIGH){
34
35     //Caso en el que la posicion del motor se encuentra al interior del
36     //intervalo entre 0 y 150 grados.
```

```
  if (GeneralPosition > 0 && GeneralPosition < 150){

    //Manual Mode
    if (Mode == LOW){

      int n = ManualMode(L_Rot, R_Rot);
      int Grades = n*LuxometerStepGrades;
      int Steps = map(Grades, 0, MaxGrades, 0, StepsPerLoop);
      Motor.step(Steps);
      //Actualiza la posicion general del motor
      GeneralPosition = GeneralPosition + Grades;
      EEPROM.update(0, GeneralPosition);
    }

    //Automatic Mode
    if (Mode == HIGH){
      if (Serial.available()){
        int X = Serial.parseInt();
        int Grades = X*LuxometerStepGrades;
        int Position = GeneralPosition + Grades;
        if (Position >= 0 && Position <= 150){
          int Step = map(Grades, 0, MaxGrades, 0, StepsPerLoop);
          Motor.step(Step);
          GeneralPosition = GeneralPosition + Grades;
          EEPROM.update(0, GeneralPosition);
        }
        else{Serial.println("Error, valor no permitido");}
      }
    }
  }

  //Caso en el que el motor se encuentra en el limite inferior de 0 grados.
  if (GeneralPosition == 0){
    L_Rot = LOW;
    //Manual Mode
    if (Mode == LOW){
      int n = ManualMode(L_Rot, R_Rot);
      int Grades = n*LuxometerStepGrades;
      int Steps = map(Grades, 0, MaxGrades, 0, StepsPerLoop);
      Motor.step(Steps);
      //Actualiza la posicion general del motor
      GeneralPosition = GeneralPosition + Grades;
      EEPROM.update(0, GeneralPosition);
    }
    //Automatic Mode
    if (Mode == HIGH){
      if (Serial.available()){
        int X = Serial.parseInt();
        int Grades = X*LuxometerStepGrades;
        int Position = GeneralPosition + Grades;
        if (Position >= 0 && Position <= 150){
          int Step = map(Grades, 0, MaxGrades, 0, StepsPerLoop);
          Motor.step(Step);
          GeneralPosition = GeneralPosition + Grades;
          EEPROM.update(0, GeneralPosition);
        }
        else{Serial.println("Error, valor no permitido");}
      }
    }
  }

  //Caso en el que el motor se encuentra en el limite superior de 150 grados.
  if (GeneralPosition == 150){
    R_Rot = LOW;

    //Manual Mode
    if (Mode == LOW){
      int n = ManualMode(L_Rot, R_Rot);
      int Grades = n*LuxometerStepGrades;
      int Steps = map(Grades, 0, MaxGrades, 0, StepsPerLoop);
```

```
        Motor.step(Steps);
        //Actualiza la posicion general del motor
        GeneralPosition = GeneralPosition + Grades;
        EEPROM.update(0, GeneralPosition);
      }
      //Automatic Mode
      if (Mode == HIGH){
        if (Serial.available()){
          int X = Serial.parseInt();
          int Grades = X*LuxometerStepGrades;
          int Position = GeneralPosition + Grades;
          if (Position >= 0 && Position <= 150){
            int Step = map(Grades, 0, MaxGrades, 0, StepsPerLoop);
            Motor.step(Step);
            GeneralPosition = GeneralPosition + Grades;
            EEPROM.update(0, GeneralPosition);
          }
          else{Serial.println("Error, valor no permitido");}
        }
      }
    }
  }

  else{
    digitalWrite(8, LOW);
    digitalWrite(9, LOW);
    digitalWrite(10, LOW);
    digitalWrite(11, LOW);
  }

}

int ManualMode(int Left, int Right){
  int n;
  if (Left == HIGH && Right == HIGH){n = 0;}
  if (Left == HIGH && Right == LOW){n = -1;}
  if (Left == LOW && Right == HIGH){n = 1;}
  if (Left == LOW && Right == LOW){n = 0;}
  return n;
}
```

2.1. Descripción

El programa contiene las librerías EEPROM y Stepper. La primera de ellas se utiliza para guardar información en la memoria EEPROM del Arduino, mientras que la segunda se utiliza para controlar el motor. Además, incluye cinco variables globales. A continuación se describe cada una:

- StepsPerLoop: Cantidad de pasos que requiere el motor para completar una vuelta de 360º. El motor 28BYJ-48 requiere de 2048 pasos.
- MotorSpeed: Rapidez del motor en RPM.
- MaxGrades: Ángulo máximo de rotación en grados. En este caso el motor tiene permitida la rotación completa.
- LuxometerStepGrades: Ángulo de rotación de la perilla del luxómetro en grados. Este valor será utilizado para realizar los giros del motor.
- GeneralPosition: Posición angular del motor, en grados.

En el programa también se definen tres funciones principales que se describirán a continuación:

- setup(): Se ejecuta una vez y su función es configurar el Arduino. Esto incluye el modo de los pines (es decir, si es modo INPUT u OUTPUT) y la velocidad del motor.
- loop(): Función que ejecuta el ciclo principal.
- ManualMode(): Función que recibe dos variables enteras, Left y Right (estas indican si hay que ejecutar el giro y en que sentido) y devuelve un número entero n que puede tomar los valores 0, 1 y -1. Estos valores se utilizan luego para no girar, girar en sentido horario y girar en sentido antihorario respectivamente.

La variable global GeneralPosition es la que posibilita limitar el movimiento del motor. Con el fin de que el Arduino recuerde el último valor de GeneralPosition, se implementa la biblioteca EEPROM. Luego de cada vuelta se actualiza GeneralPosition y utilizando la función EEPROM.update(), este dato se guarda en la memoria del Arduino, solo si el dato que

se encuentra guardado es diferente del que se busca guardar, con el fin de optimizar la vida útil de la memoria.

El código se divide en los diversos casos y modos posibles comentados en la sección anterior. Para este fin, se definen cuatro variables que serán explicadas a continuación:

- Power: Esta variable se lee del pin 2 del Arduino e indica si se encuentra apagado o encendido. La señal leída proviene del DIP switch.
- Mode: Esta variable se lee del pin 3 del Arduino e indica la selección de modo. '0' para modo manual y '1' para modo automático. La señal leída proviene del DIP switch.
- L_{Rot}: Esta variable se lee del pin 4 del Arduino e indica si se debe realizar un giro de 30º hacia la izquierda. La señal leída proviene de uno de los pulsadores.
- R_{Rot}: Esta variable se lee del pin 5 del Arduino e indica si se debe realizar un giro de 30º hacia la derecha. La señal leída proviene de uno de los pulsadores.

Dependiendo de los valores de cada una de ellas, se activa o desactiva la rotación. En la figura 5 se observa un diagrama de flujo de este proceso.

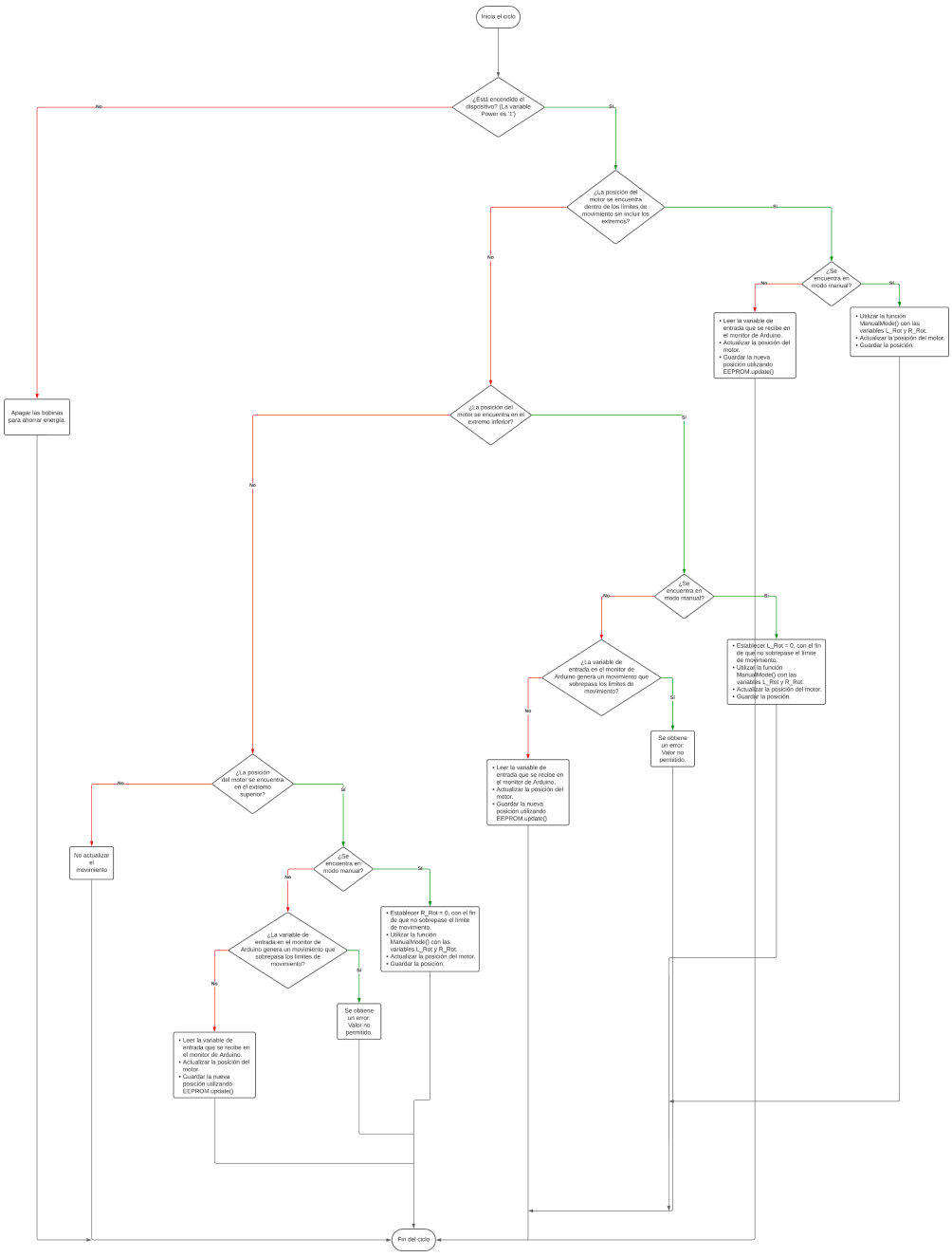


FIGURA 5. Imagen que ilustra el diseño planeado para ubicar el circuito y luxómetro.

3. Dificultades Encontradas y Sugerencias

Una de las dificultades más relevantes encontradas en cuanto al diseño eléctrico del circuito y el diseño del código fue el manejo de errores. Si bien es cierto que es una sección muy relevante, en el caso de este proyecto no fue considerada hasta cierto nivel de desarrollo. Esto ocasionó que fueran necesarios cambios en las estructuras principales del programa y del circuito. De esta forma, se ha identificado que determinar la mayor cantidad de casos posibles es de vital importancia al comienzo del desarrollo de proyectos de este estilo como también lo es realizar el análisis de los mismos desde la perspectiva de un usuario.

Se observó otra dificultad relacionada con la anterior, esta es establecer un algoritmo que permita un movimiento adecuado del motor sin exceder los límites establecidos por el luxómetro. Este aspecto es de vital importancia, ya que un mal movimiento podría dañar la perilla del luxómetro. En esencia, el mayor desafío fue idear una solución para aquellos casos en los que el dispositivo se utilizara de manera inusual, como ser desconectado espontáneamente, por ejemplo. Para abordar este problema, fue necesario utilizar la librería EEPROM,

como se mencionó anteriormente, para que el Arduino pueda almacenar en cada momento en que se actualice la posición del motor, dicha posición.

Una recomendación con respecto al diseño del código es intentar optimizar el uso de energía. Por ejemplo, se podría desconectar las bobinas del motor durante los ciclos en los que no se realice una actualización de la posición. Esta estrategia probablemente también contribuirá a disminuir el calentamiento del motor, un aspecto que observamos durante nuestras pruebas.

Además, considerando que las celdas de memoria del Arduino, donde se almacena la información de la posición del motor, tienen una vida útil de aproximadamente 100,000 escrituras/lecturas, sería ideal buscar maneras de reducir la cantidad de veces que se realiza la escritura. Una posible solución sería disminuir la carga de una de las celdas, intercambiando espacios de memoria cada cierta cantidad de iteraciones.

Por otro lado, en cuanto al diseño de las piezas para el acople, se identificaron diversas limitaciones que requieren atención y mejoras adicionales en la configuración inicial. En primer lugar, se debe tener en cuenta el aumento del grosor de las piezas impresas en 3D con el objetivo de proporcionar una mayor estabilidad y rigidez al sistema. Siguiendo esta idea, se sugiere reconsiderar la selección del material, orientándose hacia opciones metálicas, ya que estas ofrecen propiedades mecánicas superiores, garantizando una mayor robustez y vida útil del conjunto.

Finalmente, se propone configurar el diseño en forma de caja cerrada en la parte inferior, complementada por una compuerta en la parte superior del luxómetro. Esto mejorará la practicidad y eficiencia operativa del sistema.

4. Bibliografía:

1. A. Malvino and D. J. Bates, Principios de Electrónica, 7th ed. ((McGraw-Hill Interamericana, 2007), pp. 175–190.
2. M. Mano, Logica Digital y Diseño de Computadores, 1st ed. (Prentice Hall, Mexico, 1982).
3. O. B. Javier Cardona, Electrónica Digital y su Aplicación a la Instrumentación: guía de laboratorio (Facultad de Ciencias. Universidad Nacional de Colombia, 2011).
4. Arduino, Stepper Library, https://www.arduino.cc/reference/en/libraries/stepper/ (2023)
5. Arduino, EEPROM Library, https://docs.arduino.cc/learn/built-in-libraries/eeprom (2023)